

ThoughtMap v0.1 手順書

1. フォルダ構成

```
D:\ThoughtMap
├── lyrics
│   ├── 001.txt
│   ├── 002.txt
│   └── ...
├── src
│   ├── test.py
│   ├── embed.py
│   ├── similarity.py
│   ├── cluster.py
│   ├── cluster_export.py
│   ├── cluster_details.py
│   ├── cluster_map.py
│   ├── title_map.py
│   ├── author_profile.py
│   ├── author_profile_named.py
│   ├── thought_continent.py
│   ├── civilization_map.py
│   ├── civilization_pie.py
│   └── thought_search.py
└── output
    ├── lyrics_map.png
    ├── lyrics_title_map.png
    ├── lyrics_index.txt
    ├── similarity_report.txt
    ├── cluster_report.txt
    ├── lyrics_clusters.csv
    ├── cluster_map.png
    ├── cluster_distance_report.txt
    ├── author_profile.png
    ├── author_profile_named.png
    ├── thought_continent.png
    ├── civilization_map.png
    └── civilization_pie.png
```

2. Python環境

使用Python :

Python 3.13

実行時は基本的にこれを使う。

```
C:\Users\flyin\AppData\Local\Programs\Python\Python313\python.exe
```

必要ライブラリ：

```
C:\Users\flyin\AppData\Local\Programs\Python\Python313\python.exe -m pip install pandas  
matplotlib sentence-transformers umap-learn scikit-learn janome openai
```

3. 歌詞データ準備

D:\ThoughtMap\lyrics に歌詞txtを入れる。

例：

```
Algorithm Blues.txt  
Conversation Music.txt  
Community Dependency Syndrome.txt
```

中身は歌詞本文だけでOK。

4. 読み込み確認

test.py

目的：

lyricsフォルダ内のtxtが読めるか確認

実行：

```
C:\Users\flyin\AppData\Local\Programs\Python\Python313\python.exe d:\ThoughtMap\src\test.py
```

成功例：

歌詞数：100

5. 歌詞マップ生成

embed.py

目的：

歌詞をEmbedding化して2Dマップを作る

実行：

```
C:\Users\flyin\AppData\Local\Programs\Python\Python313\python.exe d:\ThoughtMap\src\embed.py
```

出力：

```
D:\ThoughtMap\output\lyrics_map.png  
D:\ThoughtMap\output\lyrics_index.txt
```

6. 曲名付きマップ生成

title_map.py

目的：

マップ上に曲名を表示する

実行：

```
C:\Users\flyin\AppData\Local\Programs\Python\Python313\python.exe d:\ThoughtMap\src\title_map.py
```

出力：

```
D:\ThoughtMap\output\lyrics_title_map.png
```

日本語が文字化けする場合：

```
import matplotlib  
matplotlib.rcParams["font.family"] = "Yu Gothic"
```

または

```
matplotlib.rcParams["font.family"] = "Meiryo"
```

7. 類似作品レポート生成

similarity.py

目的：

各曲に近い作品TOP5を出す

実行：

```
C:\Users\flyin\AppData\Local\Programs\Python\Python313\python.exe d:\ThoughtMap\src\similarity.py
```

出力：

D:\ThoughtMap\output\similarity_report.txt

8. クラスタリング

cluster.py

目的：

100曲を8つの思想クラスタに分類する

実行：

```
C:\Users\flyin\AppData\Local\Programs\Python\Python313\python.exe d:\ThoughtMap\src\cluster.py
```

出力：

D:\ThoughtMap\output\cluster_report.txt

9. CSV出力

cluster_export.py

目的：

曲名・クラスタ番号・座標をCSVに保存する

実行：

```
C:\Users\flyin\AppData\Local\Programs\Python\Python313\python.exe d:\ThoughtMap\src\cluster_export.py
```

出力：

D:\ThoughtMap\output\lyrics_clusters.csv

10. クラスタ詳細確認

cluster_details.py

目的：

各Clusterに含まれる作品一覧を見る

実行：

```
C:\Users\flyin\AppData\Local\Programs\Python\Python313\python.exe d:\ThoughtMap\src\cluster_details.py
```

11. クラスタマップ生成

cluster_map.py

目的：

クラスタ同士の距離を可視化する

実行：

```
C:\Users\flyin\AppData\Local\Programs\Python\Python313\python.exe d:\ThoughtMap\src\cluster_map.py
```

出力：

```
D:\ThoughtMap\output\cluster_map.png  
D:\ThoughtMap\output\cluster_distance_report.txt
```

12. 作者プロフィール生成

```
author_profile.py
```

目的：

各Clusterに作品が何曲あるか棒グラフ化する

実行：

```
C:\Users\flyin\AppData\Local\Programs\Python\Python313\python.exe d:\ThoughtMap\src\author_profile.py
```

出力：

```
D:\ThoughtMap\output\author_profile.png
```

13. クラスタ名設定

```
D:\ThoughtMap\output\cluster_labels.json
```

 を作る。

例：

```
{  
  "0": "制度・評価軸批評",  
  "1": "技術未来・再構築",  
  "2": "失敗・試行錯誤",  
  "3": "認識OS・情報処理",  
  "4": "社会構造・停滞批評",  
  "5": "哲学・判断・価値観",  
  "6": "内省・意味生成",  
}
```

```
"7": "因果・正当性"  
}
```

14. 名前付き作者プロフィール生成

author_profile_named.py

目的：

Cluster番号ではなく思想ラベルで作者プロフィールを表示する

実行：

```
C:\Users\flyin\AppData\Local\Programs\Python\Python313\python.exe d:  
\ThoughtMap\src\author_profile_named.py
```

出力：

D:\ThoughtMap\output\author_profile_named.png

15. 思想大陸図生成

thought_continent.py

目的：

作品点とクラスタ重心を同じマップ上に表示する

実行：

```
C:\Users\flyin\AppData\Local\Programs\Python\Python313\python.exe d:  
\ThoughtMap\src\thought_continent.py
```

出力：

D:\ThoughtMap\output\thought_continent.png

16. 思想文明マップ生成

civilization_map.py

目的：

クラスタをさらに上位概念の文明にまとめる

実行：

C:\Users\flyin\AppData\Local\Programs\Python\Python313\python.exe d:
\ThoughtMap\src\civilization_map.py

出力：

D:\ThoughtMap\output\civilization_map.png

17. 思想文明構成比

civilization_pie.py

目的：

思想文明の比率を円グラフ化する

実行：

C:\Users\flyin\AppData\Local\Programs\Python\Python313\python.exe d:
\ThoughtMap\src\civilization_pie.py

出力：

D:\ThoughtMap\output\civilization_pie.png

18. 思想検索エンジン

thought_search.py

目的：

任意の思想・テーマで近い作品を検索する

実行：

```
C:\Users\flyin\AppData\Local\Programs\Python\Python313\python.exe d:\ThoughtMap\src\thought_search.py
```

検索例：

アルゴリズム批判
主体性
失敗
共同体依存
資本主義批判

終了：

q

19. 現状できていること

歌詞100曲
↓
Embedding
↓
類似作品検索
↓
クラスタリング
↓
思想大陸図
↓
思想文明構成比
↓
思想検索

20. 現状の思想分類例

100曲時点では、以下のような構成になった。

社会文明 34%
哲学文明 23%

認識文明 15%
内省文明 12%
試行文明 8%
因果文明 5%
技術文明 3%

21. 注意点

OpenAI APIはChatGPT Plusとは別契約。

APIを使う場合：

```
$env:OPENAI_API_KEY="sk-..."
```

またはCMDなら：

```
set OPENAI_API_KEY=sk-...
```

ただし現状の主要機能はAPIなしで動作する。

APIが必要なのは主に以下。

クラスタ名の自動命名
作品群の自然言語レビュー
作者思想プロフィールの文章生成

22. 次に作れるもの

候補：

1. thought_search.py の保存機能
2. 新曲診断機能
3. 1000曲対応
4. Webアプリ化
5. 他作者との思想距離比較
6. クラスタ名のAI自動生成
7. 作品タイムライン分析

現状は `ThoughtMap v0.1` と呼べる段階。